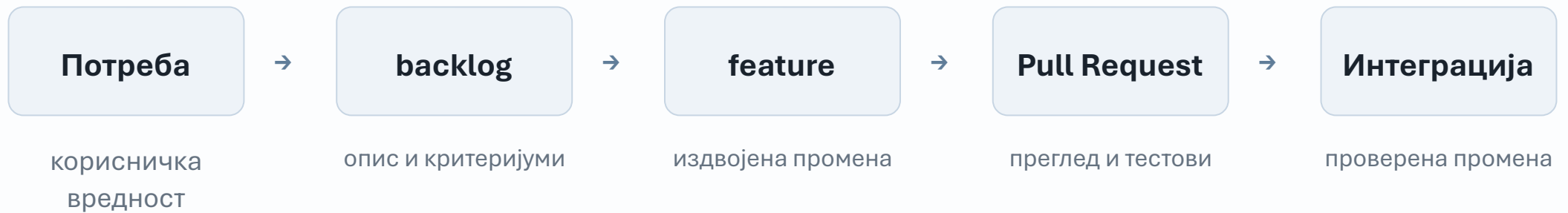


Захтеви, backlog, Git и тимски развој

Од захтева и backlog ставке до проверене промене у репозиторијуму.

Развојни ток од захтева до интеграције

Захтев, планирање рада и Git чине повезан процес.



НАПОМЕНА

Развојни ток повезује захтев, одлуке, имплементацију и доказе провере.

Структура корисничке приче

Корисничка прича наводи корисника, потребу и очекивану вредност.

**Као наставник лабораторије,
желим да резервишем слободну опрему за одређени термин,
како бих знао да је опрема доступна и спречио двоструку
резервацију.**

НАПОМЕНА

Корисничка прича има вредност када дефинише проверљиво понашање система.

Критеријуми прихватања

Сваки критеријум треба да буде проверљив тестом или демонстрационим сценаријем.

Тип сценарија	Пример критеријума
Успешан ток	Резервација се креира када је опрема доступна и период исправан.
Пословни конфликт	Систем одбија период који се преклапа са активном резервацијом.
Неисправан улаз	Почетак термина мора бити пре краја.
Гранични случај	Правило јасно дефинише термин краја и почетка термина.

Структура backlog-a

Ставка спремна за рад садржи циљ, приоритет, критеријуме и позната ограничења.

01

Product Backlog

све будуће ставке

02

Sprint Backlog

изабрани обим рада

03

Increment

реализована промена

Стања у развојном току

Статус картице одговара стварном стању посла.



НАПОМЕНА

Статус Done захтева испуњене критеријуме, успешно извршене провере и интегрисану промену.

Садржај backlog ставке

Опис, критеријуми, приоритет и подаци о одговорности чувају се уз ставку.

01

Опис

шта се тражи и ко добија вредност

02

Критеријуми

како се доказује исправност

03

Приоритет и sprint

зашто се ради сада

04

Assignee и коментари

ко ради и шта је договорено

Разлагање задатка

Контролна листа смањује ризик и чини напредак видљивим.

ПРИМЕР КОРАКА

wireframe или ток екрана

имплементација основне логике

тестови кључног правила

code review и дорада

ЕФЕКТИ

мање ризика

видљив застој

лакша провера

лакше образложење

Git историја измена

Историја развоја омогућава реконструкцију малих и смислених промена.

Појам	Улога у раду тима
Working tree	датотеке које се тренутно мењају
Staging area	избор садржаја за наредни commit
Commit	кохерентна промена са описом намере
Branch	издвојена линија развоја
Remote	удаљени репозиторијум за размену промена

Пример малог развојног корака

Команде су део договореног поступка припреме и провере измене.

```
1 git status
2 git diff
3 git add src/Application/ReservationService.cs
4 git add tests/ReservationServiceTests.cs
5 git diff --staged
6 git commit -m "Add reservation overlap rule"
7 git push -u origin feature/reservation-overlap
```

НАПОМЕНЕ

Пре commit-а прегледати припремљени diff.

Један commit треба да има једну кохерентну намеру.

Неповезане промене не иду у исти Pull Request.

Главна грана остаје стабилна за цео тим.

Pull Request и преглед измене

Преглед кода је обавезан део развојног поступка.

01

Обим

да ли промена решава тачну ставку

02

Архитектура

да ли се поштују договорене границе

03

Тестови

да ли постоји доказ провере

04

Ризици

шта се мења и шта може поћи по злу

Решавање конфликта при спајању

Тим усаглашава очекивано понашање пре завршетка спајања.

```
1 git checkout main
2 git pull
3 git checkout feature/reservation-overlap
4 git merge main
5 # решити конфликт и прегледати diff
6 git add src/Application/ReservationService.cs
7 git commit -m "Resolve reservation rule conflict"
8 dotnet test
```

НАПОМЕНЕ

Одлучити које понашање мора да остане.

Последњи корак је провера да систем и даље ради.

Force push није уобичајен ток на дељеним гранама.

Контролна тачка P1

После Вежбе 2, недеља од 19.10: проблем, backlog и развојни ток у репозиторијуму.

01

Репозиторијум и README

README и приступ свих чланова тима

02

Backlog у Tapiz Boards

приоритети и критеријуми прихватања

03

Pull request

преглед измена и образложени коментари

04

Договорени ток

од Backlog до Done, кроз Review и QA

Одговорност студента при употреби AI алата

AI алати су дозвољени уз проверљив лични допринос и разумевање решења.

ПОТРЕБНО ЈЕ РАЗГРАНИЧИТИ

сопствени рад

рад осталих чланова тима

садржај добијен спољним алатима

РЕЛЕВАНТАН ДОКАЗ РАДА

репозиторијум и историја измена

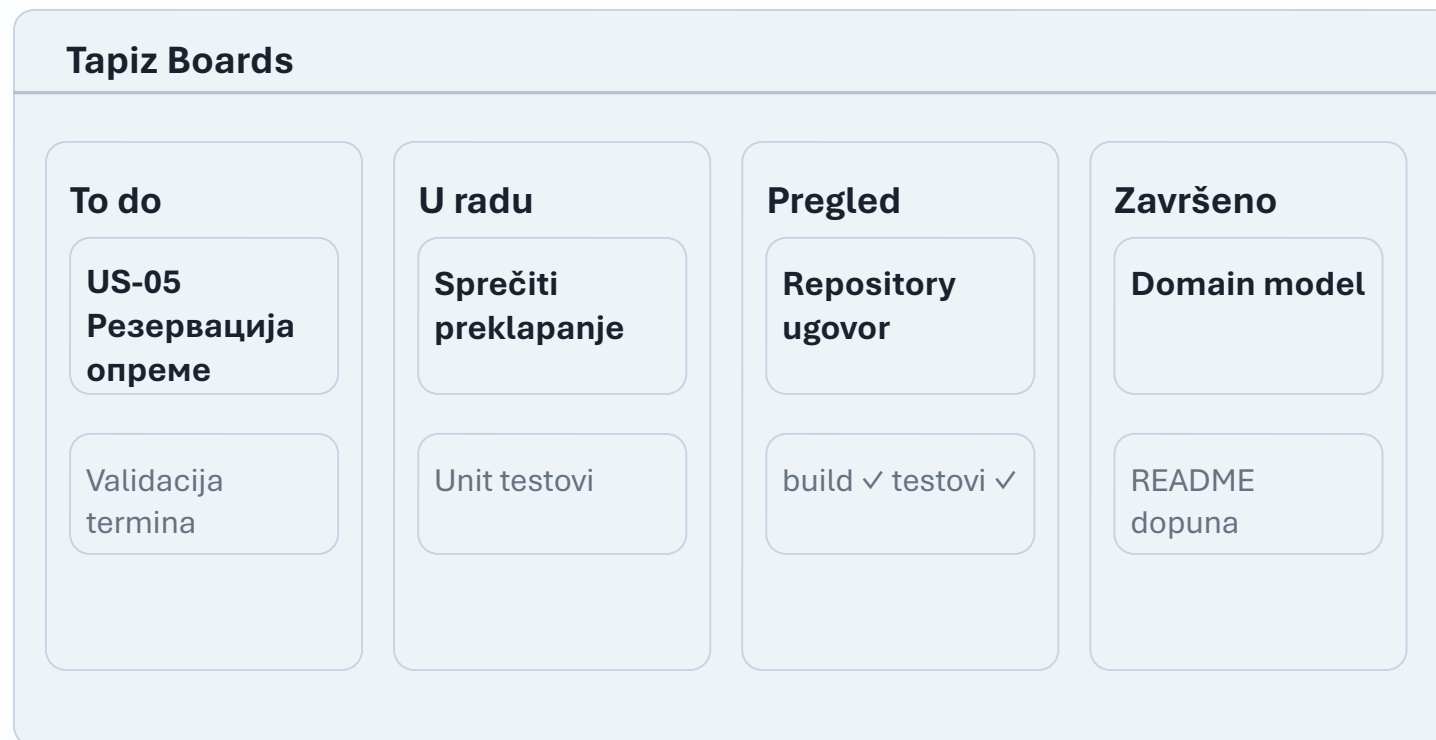
тестови и документација

евиденција употребе AI алата

самостално образложење

Tapiz Boards као договор о процесу

Табла приказује ток рада од идеје до прихваћене промене.



Како се чита табла

Колоне показују стварно стање рада, не жељу тима.

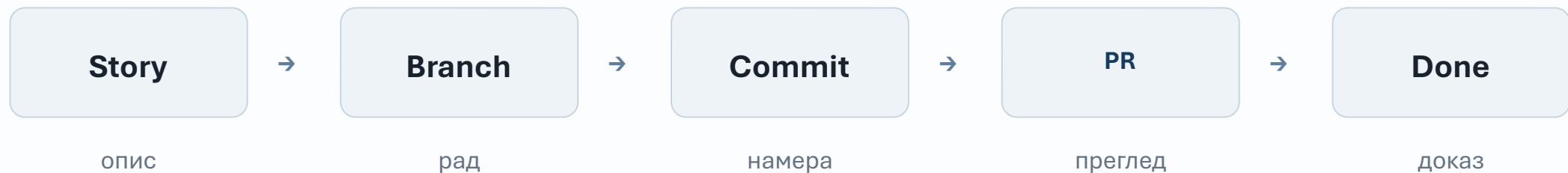
Картица треба да има опис, приоритет, assignee и критеријуме прихватања.

Веза са PR-ом и тестовима показује да је промена проверљива.

Done значи прихваћено по Definition of Done, не само померено у последњу колону.

Ток од backlog ставке до интеграције

Сваки корак оставља проверљив траг рада тима.



НАПОМЕНА

Професионални развојни ток повезује алате, договорени процес и проверу резултата.

Провера пре предаје

Тим проверава основне услове пре означавања рада као завршеног.

- 01 Да ли је ставка повезана са граном или Pull Request-ом?
- 02 Да ли критеријуми прихватања имају видљиву проверу?
- 03 Да ли diff садржи само релевантне измене?
- 04 Да ли је главна грана стабилна?
- 05 Да ли сваки члан може да објасни свој допринос?

Резиме

Циљ прве вежбе је успостављање дисциплинованог и следљивог развојног поступка.

Професионалан тимски рад почиње када се захтев може пратити до кода, теста, review-а и интеграције.

НАПОМЕНА

Ови резултати представљају основу за наредне вежбе.

Типични пропусти

Наведени пропусти нарушавају следљивост и отежавају проверу рада.

01

Нејасан Story

није јасно ко добија вредност

02

Без критеријума

не зна се када је завршено

03

Велики PR

review губи фокус

04

Без доказа

нема теста, diff-а или објашњења

Припрема за наредну вежбу

Резултати прве вежбе користе се у даљем раду на пројекту.

- 01 Написати User Story за једну ставку пројектног домена.
- 02 Додати успешне, негативне и граничне критеријуме прихватања.
- 03 Увести ставку у Tapiz ток и радити преко feature гране.
- 04 Отворити Pull Request са доказом провере.
- 05 Објаснити цео пут од захтева до интегрисане промене.